Guía de Git y GitHub para el equipo Offero

Esta guía explica cómo trabajar en equipo con ramas y Pull Requests (PR). Repositorio: `https://github.com/diegodani2030/App-Offero`

Conceptos básicos

- **Repositorio (repo):** la carpeta del proyecto con todo su historial.
- **Commit:** una "foto" de los cambios, con un mensaje que explica qué se hizo.
- **Rama (branch):** una línea de trabajo paralela. Cada persona trabaja en su rama y `main` queda limpia.
- **Pull Request (PR):** una solicitud para unir tu rama a `main`. Otra persona la revisa antes de aceptarla.
- **origin:** el nombre del repo remoto en GitHub.

Regla de oro: nadie sube directo a `main`. Todo entra por Pull Request.

Parte 1. Configuración inicial (se hace una sola vez)

1.1 Instalar Git

Descárgalo de https://git-scm.com y verifica:

```
git --version
```

1.2 Configurar tu identidad

Usa el mismo correo de tu cuenta de GitHub:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@ejemplo.com"
```

Si no lo haces, el commit falla con el error `Author identity unknown`.

1.3 Aceptar la invitación al repositorio

El dueño del repo (Diego) debe agregarte en **Settings > Collaborators > Add people**. Te llega un correo; acepta la invitación. Sin esto el `push` da error 403.

1.4 Clonar el repositorio

```
cd C:\Users\TU-USUARIO\Desktop
git clone https://github.com/diegodani2030/App-Offero.git
cd App-Offero
```

1.5 Instalar dependencias del proyecto

```
cd app-offero
npm install
npm run dev
```

Abre http://localhost:3000. Si npm da error de scripts en PowerShell:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Parte 2. Flujo de trabajo diario (para cada tarea)

Paso 1. Actualiza `main` antes de empezar

```
git checkout main
git pull origin main
```

Paso 2. Crea tu rama

```
git checkout -b feature/nombre-de-la-tarea
```

Convención de nombres (minúsculas y guiones):

Prefijo	Para qué	Ejemplo
`feature/`	Funcionalidad nueva	`feature/login-supabase`
`fix/`	Corregir un error	`fix/boton-registro`
`docs/`	Documentación	`docs/guia-git`
`style/`	Solo diseño o estilos	`style/navbar`

Paso 3. Trabaja y revisa tus cambios

```
git status
git diff
```

Paso 4. Prepara los cambios (staging)

```
git add .
```

O solo archivos concretos:

```
git add app-offero/src/app/page.js
```

Paso 5. Haz el commit

```
git commit -m "Agrega pagina de login"
```

Un buen mensaje es corto, en presente y dice **qué** hiciste. Mal: `cambios`, `arreglos`, `asdf`.

Paso 6. Sube tu rama a GitHub

La primera vez:

```
git push -u origin feature/nombre-de-la-tarea
```

Los siguientes pushes de esa misma rama:

```
git push
```

Paso 7. Crea el Pull Request

1. GitHub muestra un enlace en la terminal. Ábrelo, o entra al repo y haz clic en **Compare & pull request**.
2. Verifica que diga: base: `main` <- compare: `tu-rama`.
3. Escribe un título claro y una descripción: qué hiciste, cómo probarlo, capturas si aplica.
4. En **Reviewers**, asigna a quien revisa (Diego).
5. Haz clic en **Create pull request**.

Paso 8. Atiende los comentarios

Si el revisor pide cambios, trabaja en la **misma rama**:

```
git add .
git commit -m "Corrige observaciones del PR"
git push
```

El PR se actualiza solo.

Paso 9. Después del merge, limpia

```
git checkout main
git pull origin main
git branch -d feature/nombre-de-la-tarea
```

Parte 3. Mantener tu rama al día con `main`

Si `main` cambió mientras trabajabas:

```
git checkout main
git pull origin main
git checkout feature/tu-rama
git merge main
```

Si hay conflictos

Git avisa: `CONFLICT (content): Merge conflict in ...`. Abre el archivo en VS Code y verás:

```
<<<<<<< HEAD
tu código
=======
código que viene de main
>>>>>>> main
```

1. Elige qué conservar (VS Code ofrece botones: *Accept Current*, *Accept Incoming*, *Accept Both*).
2. Borra las marcas `<<<<<<<`, `=======`, `>>>>>>>`.
3. Termina:

```
git add .
git commit -m "Resuelve conflictos con main"
git push
```

Parte 4. Para quien acepta los PR (Diego)

4.1 Revisar un PR en GitHub

1. Ve a la pestaña **Pull requests** del repo y abre el PR.
2. En **Files changed** revisa el código. Puedes comentar líneas específicas.
3. Para aprobar o pedir cambios: botón **Review changes** > *Approve* o *Request changes* > **Submit review**.

4.2 Probar el PR en tu computador (recomendado)

```
git fetch origin
git checkout feature/nombre-de-la-rama
cd app-offero
npm install
npm run dev
```

Prueba que funcione y vuelve a `main` cuando termines:

```
git checkout main
```

4.3 Aceptar (merge) el PR

En la parte baja del PR:

- Si dice **"This branch has no conflicts with the base branch"**, haz clic en **Merge pull request** > **Confirm merge**.
- Opciones del botón:
  - **Create a merge commit:** conserva todo el historial (la opción por defecto).
  - **Squash and merge:** junta todos los commits en uno solo (historial más limpio, recomendado).
  - **Rebase and merge:** reescribe el historial linealmente (más avanzado).
- Después haz clic en **Delete branch** para borrar la rama ya unida.

4.4 Si el PR tiene conflictos

Pide a la persona que actualice su rama (Parte 3) y vuelva a hacer `git push`. También puedes usar el botón **Resolve conflicts** de GitHub para conflictos simples.

4.5 Actualizar tu `main` local después del merge

```
git checkout main
git pull origin main
```

4.6 Proteger `main` (se hace una vez)

En el repo: **Settings > Branches > Add branch protection rule**.

- Branch name pattern: `main`
- Activar **Require a pull request before merging**
- Activar **Require approvals** (1 aprobación)

Así nadie puede subir directo a `main` por error.

Parte 5. Comandos útiles (chuleta)

Comando	Qué hace
`git status`	Muestra el estado de tus archivos
`git branch`	Lista tus ramas (la actual tiene `*`)
`git branch -a`	Lista también las ramas remotas
`git checkout nombre-rama`	Cambia de rama
`git checkout -b nombre-rama`	Crea una rama y se cambia a ella
`git add .`	Prepara todos los cambios
`git commit -m "mensaje"`	Guarda los cambios
`git push`	Sube commits a GitHub
`git pull`	Baja cambios de GitHub
`git fetch origin`	Descarga info de ramas sin mezclar nada
`git log --oneline -5`	Últimos 5 commits
`git diff`	Cambios aún sin preparar
`git restore archivo`	Descarta cambios de un archivo (no se pueden recuperar)
`git restore --staged archivo`	Saca un archivo del `git add`
`git stash` / `git stash pop`	Guarda cambios temporalmente / los recupera
`git branch -d nombre-rama`	Borra una rama local ya unida

Parte 6. Errores comunes

Error	Causa	Solución
`Author identity unknown`	No configuraste nombre y correo	Parte 1.2
`403` / `Permission denied`	No eres colaborador, o no aceptaste la invitación	Parte 1.3
`rejected ... non-fast-forward`	La rama remota tiene commits que tú no tienes	`git pull` y luego `git push`
`nothing to commit`	No hay cambios, o olvidaste `git add .`	`git status`
`Your local changes would be overwritten`	Cambios sin guardar al cambiar de rama	`git add .` + `git commit`, o `git stash`

Error	Causa	Solución
`npm : No se puede cargar el archivo npm.ps1`	PowerShell bloquea scripts	Comando de la Parte 1.5
Hice commits en `main` por error	No creaste rama antes	`git branch feature/mi-rama` y luego `git checkout feature/mi-rama`; avisa a Diego para reparar `main`

Parte 7. Reglas del equipo

1. Nunca subir directo a `main`.
2. Una rama por tarea; ramas cortas y enfocadas.
3. Hacer `git pull origin main` antes de crear una rama.
4. Commits pequeños con mensajes claros.
5. **Nunca subir secretos:** el archivo `.env.local` (claves de Supabase) no se sube. Está en `.gitignore`; confirma con `git status` antes de cada commit.
6. No subir `node_modules` ni `.next`.
7. Todo PR lo revisa otra persona antes de unirse.
8. Si algo sale mal, **no borren nada**: pregunten antes. Casi todo en Git se puede arreglar.

Resumen en 10 comandos

```
git checkout main
git pull origin main
git checkout -b feature/mi-tarea
# ... trabajar ...
git status
git add .
git commit -m "Describe lo que hiciste"
git push -u origin feature/mi-tarea
# crear el Pull Request en GitHub
# después del merge:
git checkout main
git pull origin main
```